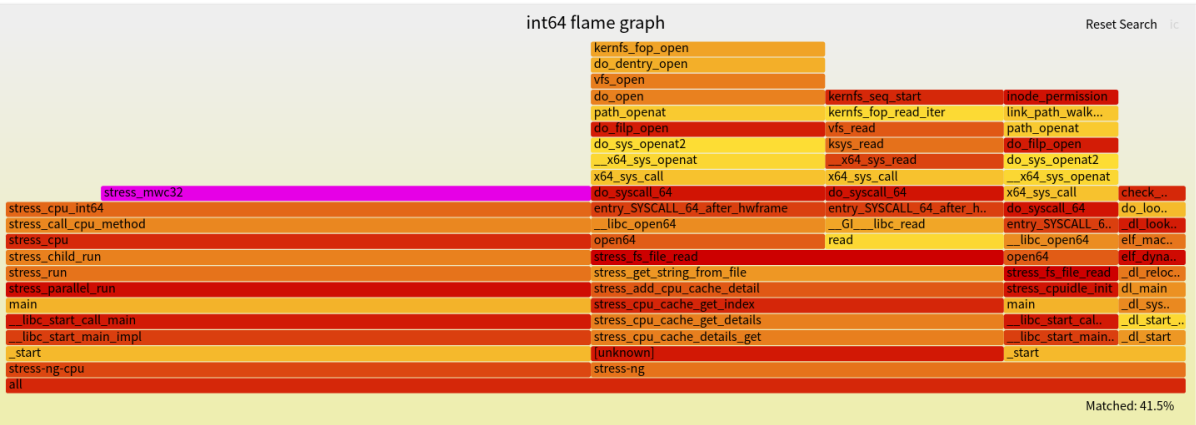


火焰图实验结果与热点分析

a. 火焰图嵌入与热点函数（占比最大函数栈）

图中紫色块就是热点函数

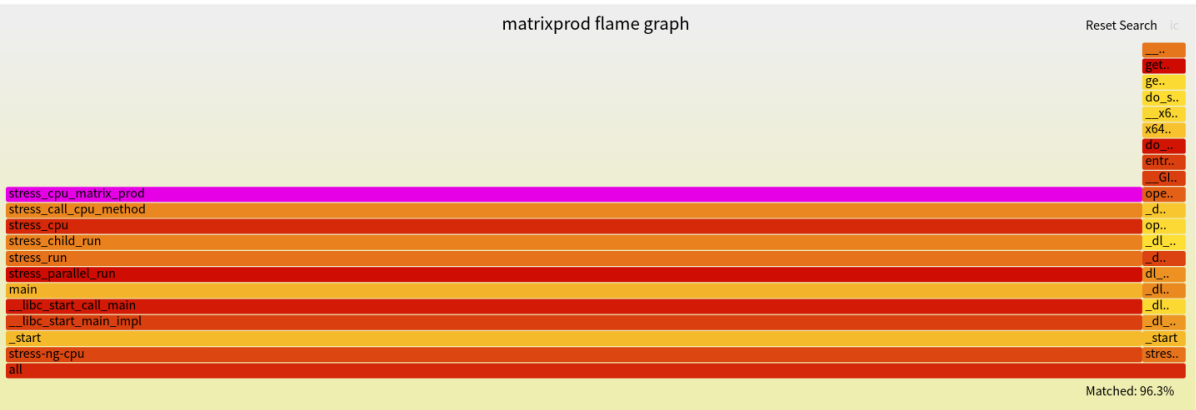
1. 纯整数计算 `int64`



热点函数（占比最大函数栈）为：

stress-ng-cpu -> main -> stress_parallel_run -> stress_run -> stress_child_run -> stress_cpu -> stress_call_cpu_method -> stress_cpu_int64 -> stress_mwc32 该栈约占 41.53%。

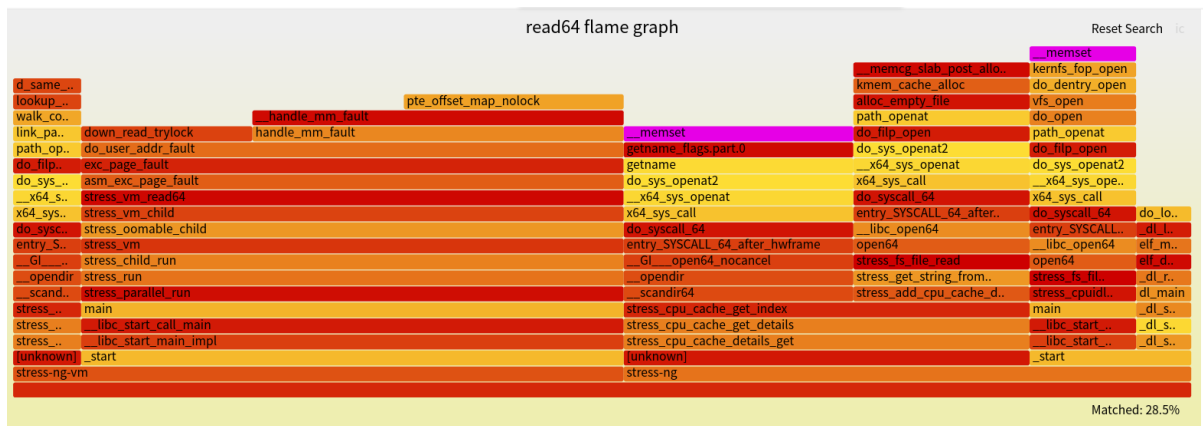
2. 矩阵计算 `matrixprod`



热点函数（占比最大函数栈）为：

stress-ng-cpu -> main -> stress_parallel_run -> stress_run -> stress_child_run -> stress_cpu -> stress_call_cpu_method -> stress_cpu_matrix_prod 该栈约占 96.32%，末端热点函数是 `stress\_cpu\_matrix\_prod`。

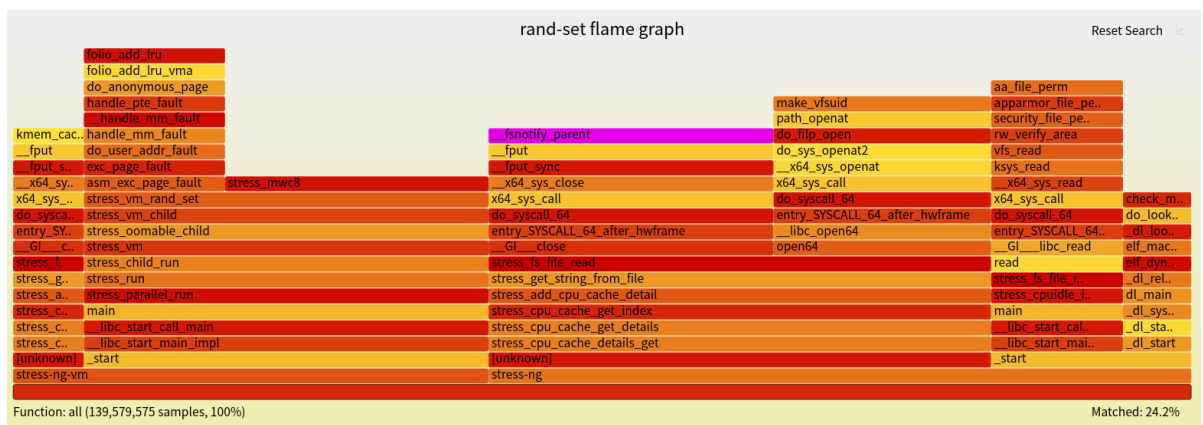
3. 连续访存 `read64`



热点函数（占比最大函数栈）为：

stress-ng -> [unknown] -> stress_cpu_cache_details_get -> stress_cpu_cache_get_details -> stress_cpu_cache_get_index -> ___scandir64 -> ___opendir -> ___GI___open64_nocancel -> entry_SYSCALL_64_after_hwframe -> do_syscall_64 -> x64_sys_call -> ___x64_sys_openat -> do_sys_openat2 -> getname -> getname_flags.part.0 -> ___memset
该栈约占 19.50%

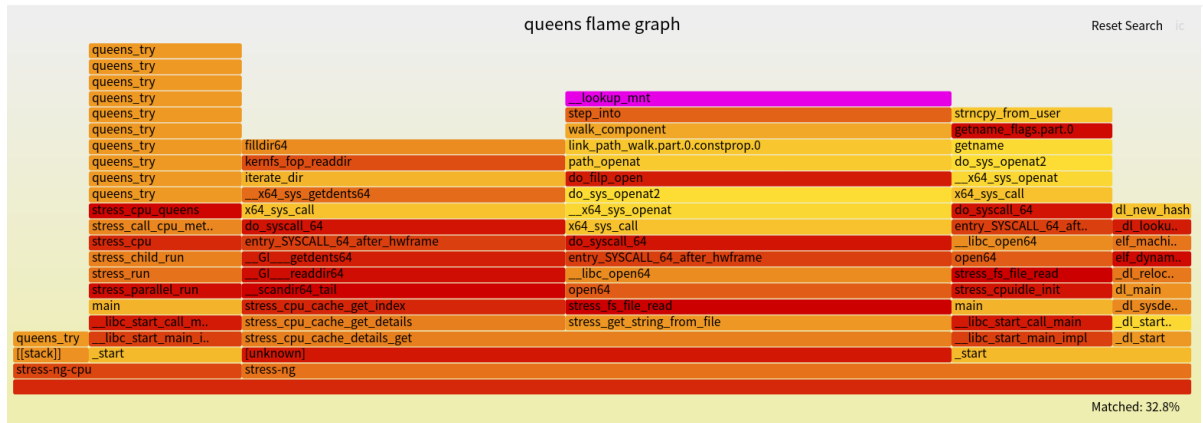
4. 随机访存 `rand-set`



热点函数（占比最大函数栈）为：

stress-ng -> [unknown] -> stress_cpu_cache_details_get -> stress_cpu_cache_get_details -> stress_cpu_cache_get_index -> stress_add_cpu_cache_detail -> stress_get_string_from_file -> stress_fs_file_read -> ___GI___close -> entry_SYSCALL_64_after_hwframe -> do_syscall_64 -> x64_sys_call -> ___x64_sys_close -> ___fput_sync -> ___fput -> ___fsnotify_parent
该栈约占 24.18%

5. 分支密集型 `queens`



热点函数（占比最大函数栈）为：

stress-ng -> [unknown] -> stress_cpu_cache_details_get -> stress_get_string_from_file -> stress_fs_file_read -> open64 -> __libc_open64 -> entry_SYSCALL_64_after_hwframe -> do_syscall_64 -> x64_sys_call -> __x64_sys_openat -> do_sys_openat2 -> do_filp_open -> path_openat -> link_path_walk.part.0.constprop.0 -> walk_component -> step_into -> __lookup_mnt，该栈约占 32.78%，主要对应 `stress-ng` 启动/探测阶段的文件打开和路径/挂载点查找。

b. 两种不同负载火焰图对比：`matrixprod` 与 `rand-set`

这里选择 matrixprod 和 rand-set 两种负载进行对比。`matrixprod` 代表计算密集型负载，rand-set 代表随机访存密集型负载，二者的火焰图形态差异最明显。

matrixprod 的火焰图呈现明显的“尖塔”形态：matrixprod 的火焰图宽度高度集中，其他路径非常窄。

热点函数（占比最大函数栈）为：

stress-ng-cpu -> main -> stress_parallel_run -> stress_run -> stress_child_run -> stress_cpu -> stress_call_cpu_method -> stress_cpu_matrix_prod
该栈约占 96.32%

这说明 `matrixprod` 的 CPU 时间几乎全部集中在矩阵乘法主循环中，CPU 时间主要集中在 stress_cpu_matrix_prod 函数中。程序绝大部分时间都在执行同一段代码，火焰图呈现典型的“尖塔”形态。它的主要瓶颈更偏向用户态计算，包括算术执行单元吞吐、流水线执行效率以及循环代码的执行效率。由于 CPU 时间位于用户态矩阵乘法计算函数中，没有明显的系统调用热点，仅有少量 open / do_syscall_64 等启动或文件访问相关系统调用。

rand-set 的火焰图的宽度更分散。它的热点函数（占比最大函数栈）为：

stress-ng -> [unknown] -> stress_cpu_cache_details_get -> stress_cpu_cache_get_details -> stress_cpu_cache_get_index -> stress_add_cpu_cache_detail -> stress_get_string_from_file -> stress_fs_file_read -> __GI__close -> entry_SYSCALL_64_after_hwframe -> do_syscall_64 -> x64_sys_call -> __x64_sys_close -> __fput_sync -> __fput -> __fsnotify_parent

该栈约占 24.18%，主要来自 `stress-ng` 启动/探测阶段的系统信息读取和文件关闭路径。与此同时，`rand-set` 的主负载相关热点还包括：

```
stress-ng-vm -> main -> stress_parallel_run -> stress_run -> stress_child_run -> stress_vm ->
stress_oomable_child -> stress_vm_child -> stress_vm_rand_set -> stress_mwc8
```

以及：

```
stress_vm_rand_set -> asm_exc_page_fault -> exc_page_fault -> do_user_addr_fault ->
handle_mm_fault -> __handle_mm_fault -> handle_pte_fault -> do_anonymous_page ->
folio_add_lru_vma -> folio_add_lru
```

CPU 时间主要分布在 `stress_vm_rand_set` 用户态随机访存函数以及 `asm_exc_page_fault`、`handle_mm_fault` 等内核态缺页处理函数上，同时伴随少量 `do_syscall_64`、`read`、`open64` 等系统调用。其 CPU 时间被分散到用户态随机访存、内核内存管理和系统调用等多个路径，因此火焰图更加扁平（）热点分散。而 `matrixprod` 的循环结构和访问模式更稳定，CPU 可以长时间执行同一段计算代码，所以热点集中。

由上面的分析可以看出：

计算密集型负载由于程序执行高度集中，数据局部性较好，CPU 后端执行单元持续工作，CPU 时间集中消耗在计算函数中。通常表现为热点集中，呈现尖塔形态。

访存密集型：随机访存破坏缓存局部性、流水线频繁停顿、TLB 和页表管理开销增加，执行时间被分散到访存、异常处理、系统调用等多个函数中。访存密集型负载通常呈现热点分散，更加扁平。

c. 内核态函数出现原因及性能影响

程序运行时，凡是涉及到 IO 读写、内存分配等硬件资源的操作时，往往不能直接操作，而是通过一种叫系统调用的过程，让程序陷入到内核态运行，然后内核态的 CPU 执行有关硬件资源操作指令，得到相关的硬件资源后在返回到用户态继续执行，之间还要进行一系列的数据传输，在这一内核态和用户态的切换过程中，会触发内核态函数的运行。

此外，下面这两种情况也会出现内核态函数的调用

异常：当 CPU 正在执行运行在用户态的程序时，突然发生某些预先不可知的异常事件，如缺页异常，触发从当前用户态执行的进程转向内核态执行相关的异常事件；

外设中断（硬中断）：当外围设备完成用户的请求操作后，会像 CPU 发出中断信号，此时，CPU 就会暂停执行下一条即将要执行的指令，转而去执行中断信号对应的处理程序，如果先前执行的指令是在用户态下，则发生从用户态到内核态的转换。

其中关于系统调用入口的内核态函数 `__x64_sys_xxx` 都会出现，因为进行压测时，都会打开文件，读取配置，查询 CPU 信息，创建线程，初始化运行环境。用户态进入内核执行 `open/read/close` 等系统调用。

关于 VFS 文件系统的内核态函数：由于需要启动阶段读取 `/proc`、`/sys` 信息，所以会调用

__x64_sys_openat、do_filp_open、path_openat、vfs_open。

关于 kernfs/sysfs 的内核态函数:kernfs_fop_open、kernfs_fop_read_iter、iterate_dir、filldir64, 用于获取 CPU、Cache、拓扑等系统信息

关于调度器与定时器的内核态函数:scheduler_tick、task_tick_fair、hrtimer_interrupt、sysvec_apic_timer_interrupt, 由周期性时钟中断、CPU 时间统计、任务调度引起

以上几种函数主要反映 程序初始化、系统调用、系统信息探测、调度器和 perf 采样 等通用内核开销, 对性能影响较小。

`int64`和`matrixprod`两类负载的内核态函数主要是于系统调用入口的内核态函数, 对性能影响较小。

运行 read64 时需要使用 --vm 分配大块匿名虚拟内存; 内核进入 handle_mm_fault, 查找 VMA、检查页表、建立 PTE 映射, 并进行内存 cgroup 统计

```
(tress_vm_read64 -> asm_exc_page_fault -> exc_page_fault
-> do_user_addr_fault -> handle_mm_fault -> __handle_mm_fault
-> pte_offset_map_nolock)
```

当 read64 第一次连续访问某些尚未建立映射的虚拟页时, CPU 触发 Page Fault;

down_read_trylock、lock_vma_under_rcu、mt_find 反映缺页处理过程中的 VMA 查找和同步控制。

```
(stress_vm_read64 -> asm_exc_page_fault -> exc_page_fault
-> do_user_addr_fault -> down_read_trylock)
```

性能影响上, page fault 会导致 CPU 从用户态陷入内核态, 暂停当前流水线, 执行页表检查、VMA 查找和映射建立; 这会增加延迟并扰动 TLB/cache。

`rand-set`会在大内存区域内按随机位置写入数据。随机写会不断触碰分散的虚拟页, 如果这些页尚未映射或需要写时分配, 内核就要进入缺页处理 (asm_exc_page_fault、exc_page_fault、do_user_addr_fault、handle_mm_fault、__handle_mm_fault); 分配匿名页、建立 PTE, 并把新页加入 LRU 等内存管理结构 (`handle\_pte\_fault`、`do\_anonymous\_page`、`folio\_add\_lru\_vma`、`folio\_add\_lru`)

性能影响上, 随机访存不仅让用户态等待数据返回, 还会让内核频繁参与页表和物理页管理, 导致 CPU 周期分散到 `handle\_mm\_fault`、`do\_anonymous\_page`、`folio\_add\_lru` 等路径中。这些操作会消耗大量 CPU 周期和内存带宽, 使 CPU 时间分散到多个内核态函数中。

`queens`的主负载是 N 皇后递归搜索和回溯, 较宽内核路径并不是求解 N 皇后本身需要设备或文件系统, 而是 `stress-ng` 启动/探测时扫描系统目录、读取 CPU/cache 信息产生的。

性能影响上, 这些系统调用路径会增加启动阶段开销, 并在短时间采样中占据可见宽度; 但对 `queens\_try` 递归搜索的长期执行性能影响有限。